3. **"ID k" token stream.** We scan r left-to-right for tokens of the form "`ID` k" and record the first occurrence of each k.
4. **Longest comma run.** We find the longest comma-separated integer run anywhere in r and normalize it.

These patterns capture the most common surface forms we observe in practice and are exactly those used in our implementation.

Exact-sorting criterion. If the sorted-block candidate A_{full} exists and is a permutation of all n IDs (after normalization), we declare *extraction success*. The instance counts as *exactly sorted* iff $A_{\text{full}} = B$. Missing or malformed cases are treated as incorrect in the exact-sorting accuracy.

Best-of-candidates policy and anchors (for substring analysis). For contiguous-substring analysis (§C) we consider *all* non-empty candidates $\mathcal{A} = \{A_1, \ldots, A_K\}$ formed by items 2–4 above (and A_{full} if present). When multiple candidates tie on the score defined next, we break ties by method priority:

$$\texttt{sorted_block_full} \prec \texttt{ids_braces} \prec \texttt{id_stream} \prec \texttt{comma_run}.$$

For the chosen best candidate we additionally report an *anchor* label describing where the matched block sits inside the candidate: `start` (run begins at position 1), `end` (run ends at position $|A_k|$), `both` (the candidate equals the block), or `neither`.

Safeguards and edge cases. We ignore any integers not present in the prompt table; repeated IDs are dropped after the first mention; candidates that normalize to the empty list are discarded. These choices make the procedure robust to extraneous numbers and minor formatting quirks without conferring credit for unseen IDs.

Complexity. Regex scans and candidate construction are $O(|r|)$; all subsequent scoring (§C) is linear in the candidate length(s).

C Contiguous LCS (longest common substring) metric

Definition. Given a candidate $A = (a_1, \ldots, a_m)$ and the ground-truth order $B = (b_1, \ldots, b_n)$ (IDs unique), define the position map $\text{pos}(b_\ell) = \ell$. Let $p_t = \text{pos}(a_t)$ for $t = 1, \ldots, m$ (these indices exist by construction after normalization). The *contiguous* LCS length between A and B is

$$\text{LCS_len}(A, B) \;=\; \max_{1 \le i \le j \le m} \Big\{ (j - i + 1) \;\Big|\; \exists\, t \text{ s.t. } \forall\, u \in \{0, \ldots, j - i\},\; p_{i+u} = t + u \Big\}. \quad (1)$$

Intuitively, (1) is the length of the longest block in A that appears *contiguously* in B; because IDs in B are unique, this is exactly the standard longest common *substring*.

Instance score and reporting. From the set of extracted candidates $\mathcal{A} = \{A_1, \ldots, A_K\}$ we take the best match

$$L^\star \;=\; \max_{k \in [K]} \text{LCS_len}(A_k, B), \qquad \text{LCS-frac} \;=\; \frac{L^\star}{|B|} \;=\; \frac{L^\star}{n}.$$

We report the mean LCS-fraction over instances with $L^\star > 0$ and a separate *coverage* rate (% of instances with any non-empty match). Exact-sorting accuracy and LCS are shown in Fig. 8.

Anchors. For the candidate achieving $L^\star$ (after the tie-break above), we also emit the anchor label `start/end/both/neither` based on whether the best contiguous block touches the candidate's boundaries. Anchors help diagnose whether the model's claimed "sorted" list appears as a leading or trailing segment versus a mid-sequence fragment.

D LIS Task Response Length and Entropy

Figure 7 shows response length during training on the LIS task. Under the answer-only reward, responses quickly shorten, while the format reward preserves lengths close to those of the base model.

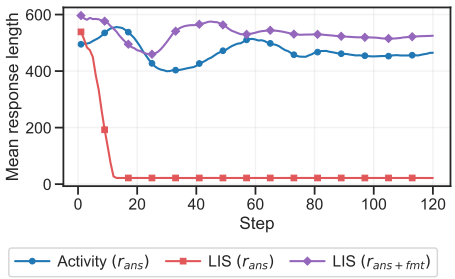

Figure 7: Response length during training

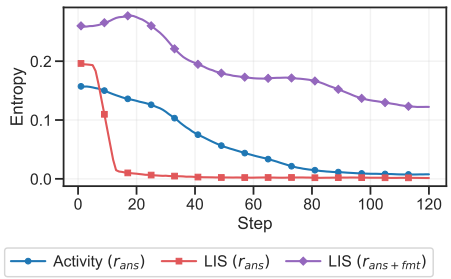

Figure 8: Entropy during training

Figure 8 further compares entropy during training. Entropy drops sharply for r_{ans}, mirroring the collapse in response length, whereas $r_{\text{ans+fmt}}$ maintains higher entropy throughout training. Despite these qualitative differences, both RLVR-trained policies converge to similar levels of Acc_{ans} and Acc_{ids}, suggesting reliance on comparable heuristics expressed through different generation styles. For example, with $r_{\text{ans+fmt}}$ nearly all responses contain Python code (100%, compared to 35.1% for the base model), yet the model does not execute the code step by step to derive the answer. This observation echoes findings by Shao et al. [7], who show that RLVR can amplify spurious reward signals in the Qwen family, encouraging models to emit structured but ultimately superficial outputs.

E Random-Forest Regression for LIS: Features, Protocol, and Training

Goal. We analyze the model's implicit decision rule by regressing its numeric answer, $y =$ `pred_lis_len`, on interpretable features computed *only* from the input values $(v_1, \ldots, v_n)$.

Data and target. From the JSONL logs we pool all stochastic runs k for each instance (`sample_idx`). The regression target is the model's emitted `\answer{\cdot}` value; no ground-truth labels are used as features.

Group split to avoid leakage. We perform a single group hold-out split with GroupShuffleSplit (test size $\approx 25\%$) using `sample_idx` as the group key. Thus, all k replicates of a given instance are kept together in train or test; there is no overlap of `sample_idx` between splits.

Feature set (input-only). Let n be the sequence length and $\Delta_i = v_{i+1} - v_i$ for $i = 1, \ldots, n-1$. We compute:

- **Global scale/dispersion:** n, $\min(v)$, $\max(v)$, range $= \max - \min$, mean, std, quartiles q_{25}, q_{50}, q_{75}, `uniq_ratio` $= |\{v\}|/n$, and `dup_ratio` $= 1 -$ `uniq_ratio`.
- **Adjacent order (local trend):** `adj_inc_ratio` $= \frac{1}{n-1}\sum \mathbb{1}[\Delta_i > 0]$, `adj_dec_ratio` $= \frac{1}{n-1}\sum \mathbb{1}[\Delta_i < 0]$, `adj_eq_ratio` $= \frac{1}{n-1}\sum \mathbb{1}[\Delta_i = 0]$, `pos_delta_mean/std` on $\{\Delta_i > 0\}$, `neg_delta_mean/std` on $\{\Delta_i < 0\}$, and `sign_change_ratio` = fraction of sign flips in $(\Delta_1, \ldots, \Delta_{n-1})$.
- **Pairwise order (global monotonicity):** `pair_inc_ratio` $= \frac{1}{\binom{n}{2}}|\{i < j : v_j > v_i\}|$, `inversion_ratio` $= \frac{1}{\binom{n}{2}}|\{i < j : v_j < v_i\}|$, `tau_like` $=$ `pair_inc_ratio` $-$ `inversion_ratio` (Kendall-tau proxy ignoring ties).
- **Runs and structure:** `max_inc_run` = longest strictly increasing contiguous run, `max_dec_run` = longest strictly decreasing run, `num_monotone_runs` = number of contiguous monotone segments, `n_local_max/min` = counts of strict local maxima/minima, `record_highs` (new maxima count), `record_lows` (new minima count).
- **Heuristic LIS approximations (length only):**
  - `greedy_len`: left-to-right "append if v_i increases" record-high count.
  - `greedy_rev_len`: same on the reversed sequence.

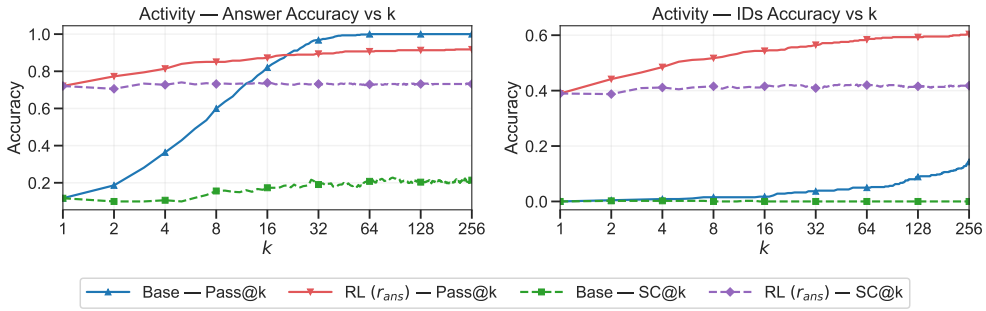

Figure 9: Performance comparison of Base and RLVR(r_{ans}) on the *Activity* task with the Llama-3.1-8B model. Left: numeric answer accuracy (Acc_{ans}) vs. k. Right: ID sequence accuracy (Acc_{ids}) vs. k.

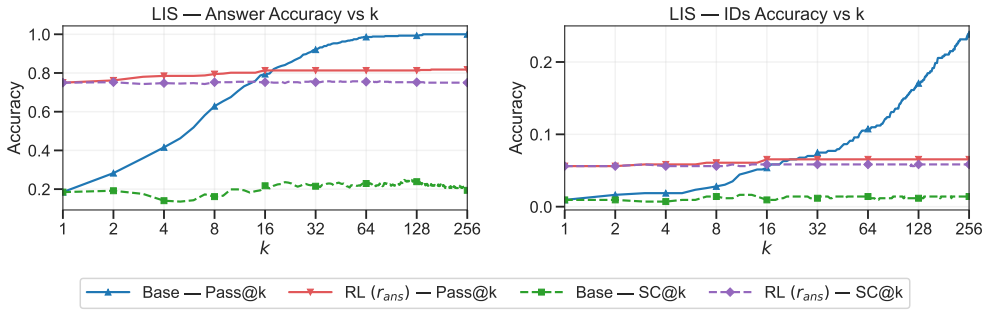

Figure 10: Performance comparison of Base and RLVR(r_{ans}) on the *LIS* task with the Llama-3.1-8B model. Left: numeric answer accuracy (Acc_{ans}) vs. k. Right: ID sequence accuracy (Acc_{ids}) vs. k.

  - `beam2`, `beam3`: beam-limited LIS lengths with beam $B \in \{2, 3\}$.
  - `budget1`, `budget2`: greedy with $s \in \{1, 2\}$ backtracks (replace tail and truncate at most s times).
- **Patience-sorting descriptors (no direct LIS leakage):** From the patience *tails* vector t, use `tail_mean`, `tail_std`, `tail_iqr`, and `tail_slope` (OLS slope of t vs. index).
- **Reference baseline:** `rand_lis_baseline` $= 2\sqrt{n}$ (typical LIS scale under random permutations).

Pre-processing. We replace $\pm\infty$ with NaN and drop rows with missing feature values. Metadata such as k or `log2_k` are never used. Standardization is unnecessary for tree ensembles.

Model and hyperparameters. We use a Random-Forest Regressor with `n_estimators`=800, `min_samples_leaf`=2, `max_features`=`sqrt`, `random_state`=42, and `n_jobs`=-1.

Evaluation and Top-K selection. We report R^2 and MAE on the held-out group-split test fold. To obtain a compact, interpretable subset, we rank features by RF importance (fit on train) and sweep $K \in \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 15, 18, 20, 25, \ldots\}$. We pick the smallest K whose test performance is within $(\Delta R^2, \Delta\text{MAE}) = (0.01, 0.02)$ of the full model. We also provide (i) a log-scaled histogram of test residuals and (ii) a Top-K curve (test R^2 and MAE vs. K).

F Llama Model Performance

Figure 9 reports results on the *Activity Scheduling* task, and Figure 10 shows results on *LIS* using the *Llama-3.1-8B* model. The LLaMA model attains higher overall accuracy on both tasks, as measured by SC@256. However, the relative trends between the base and RLVR-trained variants (e.g., under r_{ans}) closely mirror those observed with the Qwen model family.